Sarcasmometer using Sentiment Analysis and Topic Modeling

Namrata Bhan
Department of Information Technology,
Dwarkadas J. Sanghvi College of Engineering.
Mumbai, India.
namrata.bhan05@gmail.com

Prof. Mitchell D'silva
Department of Information Technology,
Dwarkadas J. Sanghvi College of Engineering.
Mumbai, India.
dsilva.mitchell@gmail.com

***Abstract*—Sarcasm is a type of sentiment where the notion that is conveyed is contrasting to what it actually means. It is usually used to denote something that is funny or to denote anger or dislike regarding a particular situation. Sarcasm has become a part of our daily lives. It is used in various social networking sites, review posts, entertainment businesses, etc.**

However, different people have different interpretation of sarcastic texts and this leads to debatable opinions about the product which is being described. Recognizing sarcastic statements can be very useful as it enhances the efficiency of after-sales services or consumer assistance through understanding the intentions and real opinions of consumers when browsing their feedbacks or complaints. In this paper we propose a system that will measure sarcasm using tweets from Twitter. We propose different algorithms to calculate the effect of sarcasm on texts and generate a score. Different features are generated from the received tweets which helps us to generate the score. At the end, we compare the scores from different algorithms to present the most efficient way to detect sarcasm. The system also provide a separate portal to check the score of any sentence/text entered by user and determine its score using the most accurate algorithm.

Keywords—*sarcasm, n-grams, linear SVC, logistic regression, Naïve Bayes, topics, sentiments.*

I. Introduction

With the advent of revolution in technology, people are using various gadgets and devices on a large scale. When an occasion commences or a new product is launched, people start writing tweets, reviews, and comments, etc. on various social media. Social media channels have become a popular platform to discuss ideas and to interact with people worldwide. For instance, Facebook claims to have 751 million monthly active users. At the same time organizations or companies launching the product also depend on social media to get an idea about their customers' views about the product as well as the product's popularity and consequently use the review or feedback to improve their products. However, finding and verifying the legitimacy of opinions or reviews is a tedious task. Thus we need to generalize the approach to measure sarcasm.

Consider an example, "I love waking up at 7am in the morning #sarcasm". The above sentence is a perfectly normal sentence, but due to the hashtag associated with it, we come to know that it intends to convey the opposite of what is written. Thus, sarcasm can be defined as an emotion which acts as an intrusive factor in any script or text that can change the polarity of the sentence. In this paper, we extract tweets from twitter using the Twitter API and preprocess the tweets. After this phase has been completed, we make a separate dictionary containing emoticons and slang words which is used to replace the expressions in tweets using them. To measure sarcasm accurately we propose a set of features namely:

1. N-grams
2. Sentiments
3. Topics
4. Pos-tag
5. Capitalization

This system uses SentiWordNet Dictionary to assign negative and positive scores to each word and store it using its POS-ID. Using the above features, we train our topic modeler using all tweets, then it generates the features for all tweets and then train a classifier using these features. It outputs a report showing the accuracy of the classifier. The different algorithms used to train the feature sets are Linear SVC, Logistic Regression and Bernoulli Naive Bayes. At the end, the system determines the algorithm with best F-score and uses it to measure sarcasm in different texts.

978-1-5386-3852-1/17/$31.00 ©2017 IEEE

Authorized licensed use limited to: BRACT's Vishwakarma Institute Pune. Downloaded on March 04,2026 at 05:25:02 UTC from IEEE Xplore. Restrictions apply.

II. Related Work

A. *Using Machine Learning Algorithms*

This system consists of 3 data sets:

i) Training set:

In this, the tweets are checked manually and given a particular sarcasm level from 1 to 6 wherein level 1 is being not sarcastic to level 6 being highly sarcastic. This set is then used for training the system.

ii) Optimization set:

In this data set, the tweets are not checked manually, and therefore it is called as the noisy set. Here, the non-English tweets, short tweets having less than 3 words, and those which contain URL's within them are filtered out to remove the noise. This dataset is used to train the parameters.

iii) Test set:

In the test set, all the tweets are checked manually. After performing manual check the tweets are further classified as sarcastic and non-sarcastic tweets.

The tweets are tested on the basis of 4 features:

1) Sentiment-related features:

This maintains two lists of different words that are qualified as "positive words" and "negative words". First list contains words that have positive emotional content (e.g., "love", "happy", etc.) and the second list contains negative emotional content (e.g., "hate", "sad", etc.). Using these two lists, two features namely "Positive words (pw)" and "Negative words (nw)" are extracted by counting the number of positive and negative words respectively in the tweet. Verbs, adverbs and adjectives have higher emotional content than nouns; therefore positive and negative words that have the associated PoS-tag, are counted again and used to create two more features that we denote PW and NW and which denotes the number of highly emotional positive words and highly emotional negative words. It also takes emoticons into consideration.

Ratio of emotional words is calculated as follows:

$$\rho(t) = \frac{(\delta_1 PW + pw) - (\delta_2 NW + nw)}{(\delta_1 PW + pw) + (\delta_2 NW + nw)} \text{ .. Equation 1 [1]}$$

2) Punctuation-Related Features:

For each tweet, the number of exclamation marks, question marks, dots, all-capital words, quotes are counted. Another sixth feature is also included by checking if any of the words contain a vowel that is repeated more than 3-4 times (e.g. "looooove"). If such a case is found then the feature is set to true else it is set to false.

3) Lexical and Syntactic Features:

i. Count of uncommon words
ii. Presence of sarcastic text or expression
iii. Number of interjections
iv. Number of laughter's

4) Pattern-Related Features:

Here the words are divided into 2 classes, CT and GFI. CT includes the words whose content is more important and GFI includes the words whose grammatical function is more important.

$$\mathbf{Lmin \leq Length(pattern) \leq Lmax}$$

where Lmin and Lmax represent the minimal and maximal allowed length of patterns in words and Length(pattern) is the length of the pattern in words. The number of pattern lengths is NL = (Lmax − Lmin + 1). We classify the resulted patterns into NF sets where,

$$\boldsymbol{NF = NL \times NS}$$

Next, resemblance, res(p,t), is calculated as follows:

$$\text{res}(p,t) = \begin{cases} 1, & \text{if the tweet vector contains the pattern as it is, in the same order} \\ \alpha + n/N, & \text{if n words out of the N words of the pattern appear} \\ 0, & \text{if no word of the pattern appear in the tweet} \end{cases}$$

The approach used in this system is tested during cross validation and also on a test set [3].

B. *Polarity Detection*

The equation which we are using to find the polarity of a sentence is:

polars = polars + polarj .. Equation 2 [1]

The polarity for a tweet is then calculated by using the below equation:

$$\frac{\sum_{i=1}^{n} (S_i)}{\text{Number of sentences in the document}} \text{ .. Equation 3 [1]}$$

1) Verb and Adjective analysis

We use a Stanford Pos-tagger to tokenize the tweets. Then the pos-tagger classifies the words as noun, verb, adjective, adverb, etc. With the help of this we can determine the relation between different parts of the tweet. The polarity of the sarcastic tweets is calculated as follows:

Algorithm 1: **Polarity of sarcastic tweets [1]**

a. Split the document with tweets for a particular candidate into sentences.
b. Extract the opinions from each sentence by recognizing the opinion adjectives, emoticons opinion verb in conjunction with punctuation marks and hash tags.
c. Determine the polarity of each opinion verb and adjectives regarding the political issue using polarity defined in the opinion dictionary.
d. Calculate the overall polarity of each sentence using Eq.2.
e. Calculate the polarity of each document using Eq. 3.
f. The opinions used in a sentence are extracted (step b) by implementing lexical analysis in each sentence using the Algorithm 2.

Authorized licensed use limited to: BRACT's Vishwakarma Institute Pune. Downloaded on March 04,2026 at 05:25:02 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: **To extract tokenized adjectives and verbs [1]**

a. Run the POS tagger to tokenize adjectives, verbs, nouns etc.
b. Determine the adjectives, main verb and hashtags.
c. Find the subject (Noun for which the verb is used) with the help of the main verb.

C. Using Big Data Approach

In this system, a Hadoop-based framework is built that allows the user to acquire and store tweets in a distributed environment and process them for detecting sarcastic content in real time using the MapReduce programming model. Figure 1 below shows the structure of sarcasm detection.

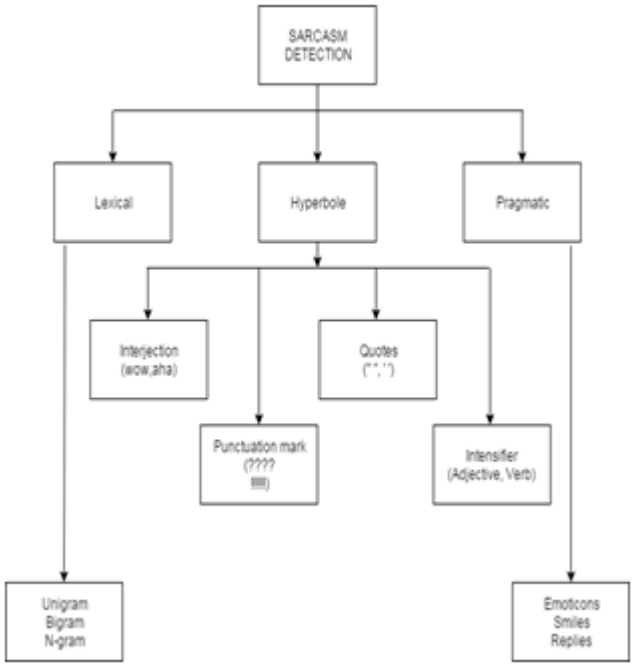

Figure 1: Breakdown structure of sarcasm detection

1) Framework

The proposed system uses the Hadoop framework to process and store the tweets streamed in real time. These tweets are retrieved from Twitter using the Twitter streaming API (Twitter4j). The Flume module is responsible for communicating with the Twitter streaming API and retrieving tweets matching certain criteria, trends or keywords. The tweets retrieved from Flume are in JavaScript Object Notation (JSON) format which is passed on to the HDFS. Oozie is a module in Hadoop that provides the output from one stage as the input to the next [2].

2) Parallel HDFS

To increase the throughput of a system and handle the massive volume of tweets, the parallel architecture of HDFS is used.

3) Sarcasm Detection Engine

To identify the sentiment of a given tweet, it passes through the MapReduce functions for sentiment classification. The tweet is classified into either a negative, positive or neutral, based on the detection engine. It uses an automated SDE which takes tweets as an input and produces the actual sentiment of the tweet as an output. Once the tweet is classified as either positive or negative, further checks are required to confirm if it has an actual positive/negative sentiment or a sarcastic sentiment.

4) POS-tagging

Parts-of-speech tagging Parts-of-speech (POS) tagging divides sentences or paragraphs into words and assigning corresponding parts-of-speech information to each word based on their relationship with adjacent and related words in a phrase, sentence, or paragraph. In this paper, a Hidden Markov Model (HMM) based POS tagger is used to identify the correct POS tag information of given words.

5) Parsing

Parsing is a process of analyzing grammatical structure, identifying its parts of speech and syntactic relations of words in sentences. When a sentence is passed through a parser, the parser divides the sentence into words and identifies the POS tag information. With the help of the POS information and syntactic relation, it forms units like subject, verb, and object, then determines the relations between these units and generates a parse tree.

6) Sentiment analysis

Sentiment analysis is a mechanism to recognize one's opinion, polarity, attitude and orientation of any target like movies, individuals, events, sports, products, organizations, locations, services, etc. To identify sentiment in given phrase, we use pre-defined lists of positive and negative words such as Sentiwordnet. It is a standard list for positive and negative English words [2]. Using the Sentiwordnet lists along with the equations, we find the sentiment score for a given phrase or sentence as follows:

$$PR=PWP/TWP \text{ .. Equation 4 [2]}$$

$$NR=NWP/TWP \text{ .. Equation 5 [2].}$$

$$\text{Sentiment Score} = PR - NR \text{ ..Equation 6 [2]}$$

where PR is the positive ratio, NR the negative ratio, PWP the number of positive words in a given phrase, NWP the number of negative words in a given phrase, and TWP the total words in given phrase.

III. Implementation

This section discusses the various stages in the implementation of the system. Fig. 2 below shows the block diagram of the proposed system to measure sarcasm using tweets. Following are the steps in the proposed system:

1) Extraction of Tweets

The system extracts the tweets using the Twitter API and the python library Tweepy. The tweets which contains "#sarcasm" are extracted and labelled as positive tweets (as they are sarcastic). The number of sarcastic tweets is around

Authorized licensed use limited to: BRACT's Vishwakarma Institute Pune. Downloaded on March 04,2026 at 05:25:02 UTC from IEEE Xplore. Restrictions apply.

25000 and the number of non-sarcastic tweets (without #sarcasm) is 110000. These tweets were extracted over a period of one month. The system uses 2 approaches while extracting the tweets wherein one is a general approach which uses just the keyword "#sarcasm" and another one is topic specific approach where another hashtag is used which contains a word related to a particular topic. For example, if we want to extract tweets that are related to IPL 2017 but are also sarcastic then we can add "#IPL2017" with "#sarcasm". In this way we get topic specific tweets that are sarcastic.

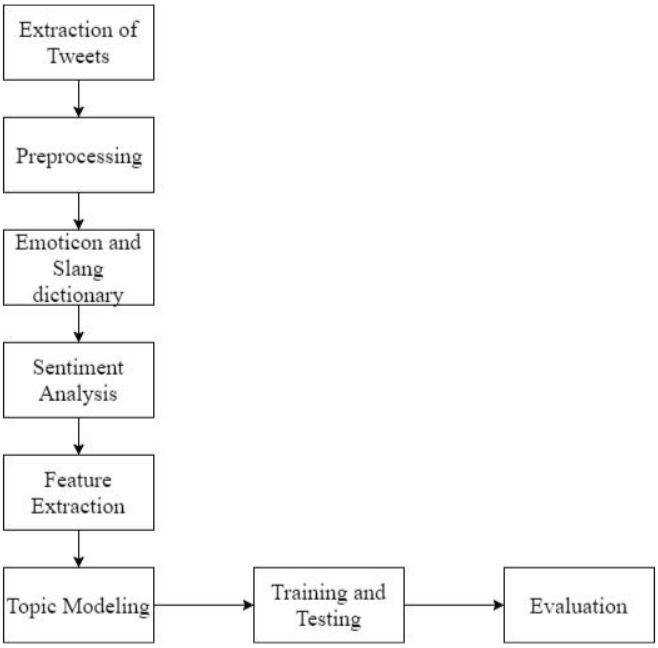

Figure 2: Block diagram of System

2) Perform Preprocessing

Twitter data is a type of real-world data and hence is often incomplete or inconsistent and likely to contain errors. Therefore, the tweets extracted from Twitter are required to be preprocessed. It removes the tweets with non-ASCII characters, tweets which are not in the English language, tweets containing URLs, etc. It also removes # (hashtags) from the sentence and any occurrences of the word sarcasm or sarcastic from the tweets. It removes the '@' tag or the friend tag. After all these steps are performed, the tweets with length equal to 3 or more are considered.

3) Emoticon and Slang Dictionary

A python file containing 54 different emoticons and slang words was developed to replace the emoticons and slang words before sending the tweets for sentiment analysis. The following examples depict how the emoticons and slang words are replaced.

Example: ":d" => "good", "yaaay" => "good", ":(" => "bad"

4) Sentiment Analysis

After making the emoticon and slang dictionary, all the emoticons and slang words were replaced as explained above and then sentiment analysis. It is the process of determining if the sentiment of a particular text is positive, negative or neutral) was performed using the following 2 options:

a. SentiWordNet
b. TextBlob

SentiWordNet is a lexical resource for opinion mining. SentiWordNet assigns to each synset of WordNet three sentiment scores: positivity, negativity, objectivity. TextBlob is a Python library for processing textual data. It provides a simple API for diving into common natural language processing (NLP) tasks such as part-of-speech tagging, noun phrase extraction, sentiment analysis, classification, translation, and more.

5) Feature Extraction

This is the most important phase in the development of the system. Before applying feature extraction algorithms, the stemming of words was performed. Stemming is the process in which the words are shortened and normalized to their stem and their tenses are ignored. For example, "cats running ran cactus cactuses cacti community communities" will be stemmed to 'cat run ran cactu cactus cacti commun'. The root of the word is preserved for better efficiency of feature extraction and to reduce redundancy. This system takes into account the features developed from N-grams, sentiments, topics, pos-tags, capitalization, etc. The features from N-grams are majorly unigrams i.e. containing one word (For example, "beautiful", "happy", etc.) and bigrams i.e. containing two words (For example, "hey there", "whats up"). Next we consider topics as features. Topics are basically words which have a high probability of appearing together. For example, "saturday", "night", "party", "fever" are mostly used together. We extract the topics from the tweets and assign separate scores to them. For example, according to our training performed words like "just what", "yay" have high occurrence in the tweets according to the scores that are generated. The sentiments from the previous step are loaded and its features are generated. For better accuracy the tweets are then splitted into 2 and 3 parts respectively and the scores are generated.

6) Topic Modeling

A topic model is a type of statistical model for discovering the abstract "topics" that occur in a collection of documents. Topic modeling is a frequently used text-mining tool for discovery of hidden semantic structures in a text body. This system uses Latent Dirichlet Allocation for modeling the topics. LDA represents documents as mixtures of topics that spit out words with certain probabilities. This system uses the Gensim python library which is a wrapper around LDA to perform topic modeling and assign scores.

7) Training and Testing

In this step, the topic modeler is trained using the tweets. This comes under the part of feature engineering. After the topic modeler is trained, a dictionary vectorizer is created and saved. Dictionary Vectorizer turns lists of mappings (dict-like objects) of feature names to feature values into Numpy arrays or Scipy sparse matrices for use with scikit-learn estimators. This vector is then sent as an input for the next step i.e. feature splitting. Feature splitting consists of steps like shuffling and

Authorized licensed use limited to: BRACT's Vishwakarma Institute Pune. Downloaded on March 04,2026 at 05:25:02 UTC from IEEE Xplore. Restrictions apply.

splitting. The tweets are shuffled for less redundancy. Splitting is performed in a way wherein 30% of the tweets are preserved as test set whereas remaining 70% are used for the training. In the next phase, we perform training, wherein we use the following algorithms to train our dataset:

a. Linear SVC (Support Vector Classifier) under SVM
b. Logistic Regression under Linear Model
c. BernoulliNB under Naive Bayes

Support vector machines (SVMs, also support vector networks) are supervised learning models with associated learning algorithms that analyze data used for classification and regression analysis. Logistic regression, or logit regression, or logit model is a regression model where the dependent variable (DV) is categorical. It covers the case of a binary dependent variable—that is, where it can take only two values, "0" and "1", which represent outcomes such as pass/fail, win/lose, alive/dead or healthy/sick. Cases where the dependent variable has more than two outcome categories may be analysed in multinomial logistic regression. Naive Bayes classifiers are a family of simple probabilistic classifiers based on applying Bayes' theorem with strong (naive) independence assumptions between the features. We use the above 3 algorithms for our study as the matrices that we get as output from dictionary vectorizer are sparse matrices. This system uses the python library Scikit-Learn which serves as a tool which provides the methods to fit our data in the classifier. After this step, the most important features found in the n-grams, sentiments, text blob sentiments and the topic model are displayed. We validate our classifier using the predict method and thus get the output for accuracy of each algorithm.

Table 1, 2 and 3 below depicts the results obtained for each of the above algorithms:

	Precision	Recall	F-Score	Support
Non-Sarcastic	0.91	0.93	0.92	35403
Sarcastic	0.63	0.58	0.60	7533
Average/Total	0.86	0.87	0.86	42936

Table 1: Linear SVC (Support Vector Classifier)

	Precision	Recall	F-Score	Support
Non-Sarcastic	0.92	0.90	0.91	35361
Sarcastic	0.57	0.66	0.61	7575
Average/Total	0.86	0.85	0.86	42936

Table 2: Logistic Regression under Linear Model

	Precision	Recall	F-Score	Support
Non-Sarcastic	0.92	0.86	0.89	35407
Sarcastic	0.50	0.67	0.57	7529
Average/Total	0.85	0.82	0.83	42936

Table 3: BernoulliNB under Naive Bayes

8) Evaluation

After the dataset has been trained, we pickle out the Dictionary Vectorizer, Classifier, Topic modeler and use it to determine the score for each sentence entered by the user. The score is positive if the sentence is sarcastic and is negative if the sentence is non-sarcastic. The formula for calculating the score (using sigmoid function) is as shown in Equation 7.

(2.0*(1.0/(1.0+e^(-score))-0.5)*100.0) .. Equation 7

where, 'score' is the value generated by the decision_function of the classifier used. Evaluation of tweets shown will differ according to the approach we use. The topic specific approach gives better accuracy as it will take into consideration only those tweets from that specific domain. This results in better accuracy in evaluation. Figure 3 shows an example where the general approach is used and a negative score is obtained as the sentence is not highly sarcastic. Figure 4 shows the output for the topic specific approach wherein the corpus used is of IPL2017, and hence the sarcasm measurement is more accurate.

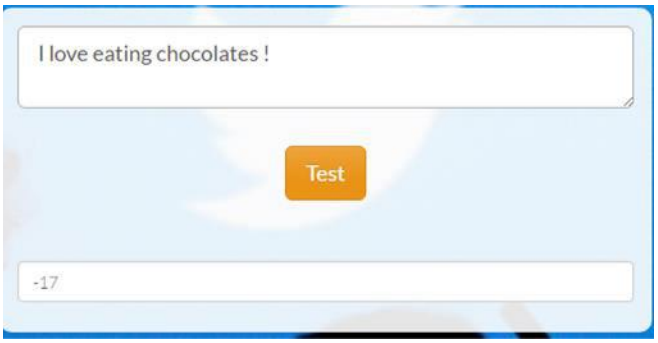

Figure 3: Output for the general approach

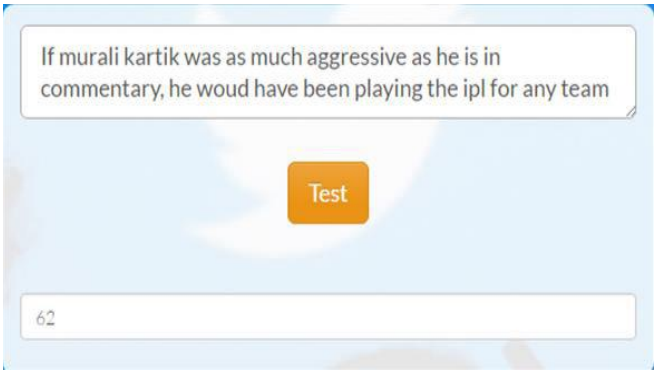

Figure 4: Output for the topic specific approach

IV. Conclusion

Sarcasm detection and analysis in social media provides in-valuable insight into the current public opinion on trends and events in real time. The Twitter API is used to stream tweets with the label #sarcasm. In this paper three algorithms namely, Naïve Bayes, Logistic Regression and Linear SVC are compared for accuracy and F-scores are generated for each algorithm respectively. Table No. 1, 2, and 3 above depict the precision, recall and F-score values for each of the algorithms. Table 4 below illustrates the comparison of the above three algorithms for their accuracy.

Authorized licensed use limited to: BRACT's Vishwakarma Institute Pune. Downloaded on March 04,2026 at 05:25:02 UTC from IEEE Xplore. Restrictions apply.

	Precision	Recall	F-Score
Naïve Bayes	0.85	0.82	0.83
Logistic Regression	0.86	0.85	0.86
Linear SVC	0.86	0.87	0.86

Table 4: Comparison of the three algorithms (Linear SVC, Logistic regression, Naive Bayes) for their accuracy

Since the F-score generated for Linear SVC and Logistic Regression are same, but the recall for Linear SVC is more than Logistic Regression, hence using Linear SVC we get better accuracy. A portal is designed to test sarcasm of texts entered by the user and is tested for both the approaches general as well as topic specific and accordingly a score is generated.

REFERENCES

[1] D. K. Tayal Sumit Yadav , Komal Gupta, Bhawna Rajput, Kiran Kumari, "Polarity Detection of Sarcastic Political Tweets", Department of Computer Science and Engineering, Indira Gandhi Delhi Technical University for Women, Kashmere Gate, Delhi, India.

[2] Roberto Gonzalez-Ibanez, Smaranda Muresan, Nina Wacholder, "Identifying sarcasm in Twitter: a Closer Look", School of Communication & Information Rutgers, The State University of New Jersey 4 Huntington St, New Brunswick, NJ 08901I.

[3] S. K. Bharti, B. Vachha, R. K. Pradhan, K. S. Babu, S. K. Jena, "Sarcastic sentiment detection in tweets streamed in real time: a bigdata approach. Department of Computer Science & Engineering, National Institute of Technology, Rourkela 769008, India.

[4] Mondher Bouazizi, Tomoaki Ohtsuk, "Sarcasm Detection in Twitter"All your products are incredibly amazing!!! – Are They Really?", Graduate School of Science and Technology,Keio University,Yokohama, Japan Email: bouazizi@ohtsuki.ics.keio.jp, Department of Information and Computer Science Faculty of Science and Technology,Keio University,Yokohama, Japan. Email: ohtsuki@ics.keio.ac.jp.

[5] Ashwin Rajadesingan, Reza Zafarani, and Huan Liu, "Sarcasm Detection on Twitter: A Behavioral Modeling Approach", Computer Science and Engineering Arizona State University {arajades, reza, huan.liu}@asu.edu.

[6] D. Davidov, O. Tsur, and A. Rappaport, "Semi-Supervised Recognition of Sarcastic Sentences in Twitter and Amazon", in Proceedings of the Fourteenth Conference on Computational Natural Language Learning, 2010, pp. 107-116.

Authorized licensed use limited to: BRACT's Vishwakarma Institute Pune. Downloaded on March 04,2026 at 05:25:02 UTC from IEEE Xplore. Restrictions apply.